Complete finite validation programs for Topic F

The human_compression project

Expanded edition

The nine files below are reproduced in full. The first three are the unchanged L014 validation programs. Two new model programs test the carry construction and corrected regular families; two complete exploratory scans are retained as diagnostics. The last two are the previously written central-lift checks. None is a finite premise of the uniform written proofs. The operational drivers and generic Lean files are inventoried separately. In particular an exhaustive bounded SmallGroups scan is not a classification proof for all odd primes.

phase236_models.py

```
#!/usr/bin/env python3
"""Exact mixed-modulus cyclotomic models; emits literal GAP permutations."""
import argparse, itertools, json, math, pathlib

PARAMETERS = [(3,3),(3,4),(3,6),(5,3),(5,5),(7,3),(11,3)]

def need(ok, msg):
    if not ok:
        raise RuntimeError('PHASE236_REJECT '+msg)

def model(p, m, corrupt=False):
    d=p-1; n=m-1; e=(n-1)//d+1; r=(n-1)%d+1
    mod=[p**(e if i<r else e-1) for i in range(d)]
    def norm(v): return tuple(x%q for x,q in zip(v,mod))
    zero=norm([0]*d); one=norm([1]+[0]*(d-1))
    def add(v,w):return norm([x+y for x,y in zip(v,w)])
    def neg(v):return norm([-x for x in v])
    def act(T,v):return norm([sum(a*b for a,b in zip(row,v)) for row in T])
    def matrix(cols):return tuple(tuple(cols[j][i] for j in range(d)) for i in range(d))
    basis=[norm([int(i==j) for i in range(d)]) for j in range(d)]
    I=matrix(basis)
    def compose(T,U):return matrix([act(T,col) for col in zip(*U)])
    def power(T,k):
        out=I
        for _ in range(k):out=compose(T,out)
        return out
    pi=[[0]*d for _ in range(d)]
    for i in range(d-1):pi[i+1][i]=1
    for i in range(d):pi[i][-1]=-math.comb(p,i+1)
    if corrupt:pi[0][-1]=0
    Z=matrix([add(act(pi,v),v) for v in basis])
    def well_defined(T):
        return all((mod[j]*T[i][j])%mod[i]==0 for i in range(d) for j in range(d))
    need(well_defined(Z),'zeta respects the mixed-modulus lattice')
    need(power(Z,p)==I,'cyclotomic p-th power')
    need(Z!=I,'nonabelian action')
    Zinv=power(Z,p-1)
    barpi=matrix([add(act(Zinv,v),neg(v)) for v in basis])
    cols=[];v=one
    for _ in range(d):cols.append(v);v=act(barpi,v)
    S=matrix(cols)
    need(well_defined(S),'conjugation respects the mixed-modulus lattice')
    need(compose(S,S)==I,'conjugation square')
    need(compose(compose(S,Z),S)==Zinv,'conjugation reverses zeta')
    for v in basis:
        w=zero;u=v
```

```
        for _ in range(p):w=add(w,u);u=act(Z,u)
        need(w==zero,'cyclotomic norm zero')
    A=list(itertools.product(*(range(q) for q in mod)))
    points=[(v,k) for v in A for k in range(p)]
    idx={v:i+1 for i,v in enumerate(points)}
    need(len(points)==p**m,'normal-form cardinality')
    backwards=[act(power(Z,(-k)%p),one) for k in range(p)]
    s=[idx[(add(v,backwards[k]),k)] for v,k in points]
    beta=[idx[(v,(k+1)%p)] for v,k in points]
    sigma=[idx[(act(S,v),(-k)%p)] for v,k in points]
    tau=[idx[(neg(v),k)] for v,k in points]
    for perm in [s,beta,sigma,tau]:need(sorted(perm)==list(range(1,len(points)+1)),'literal
     ↪ permutation')
    return dict(p=p,m=m,e=e,r=r,moduli=mod,degree=len(points),s=s,beta=beta,sigma=sigma,tau
     ↪ =tau)

def emit(path, corrupt=False):
    data=[model(p,m,corrupt=corrupt) for p,m in PARAMETERS]
    rows=[]
    for row in data:
        fields=['p:='+str(row['p']),'m:='+str(row['m']),
                'moduli:='+str(row['moduli'])]
       fields += [key+':=PermList('+str(row[key])+')' for key in ['s','beta','sigma','tau']]
        rows.append('rec('+','.join(fields)+')')
    path.write_text('PHASE236_DATA := [\n'+',\n'.join(rows)+'\n];\n')
    return [{k:v for k,v in row.items() if k not in ['s','beta','sigma','tau']} for row in
     ↪ data]

if __name__=='__main__':
    ap=argparse.ArgumentParser(description=__doc__);ap.add_argument('--out',type=pathlib.Pa
     ↪ th,required=True)
    ap.add_argument('--corrupt-cyclotomic',action='store_true');a=ap.parse_args()
    need(not a.out.exists() and not a.out.is_symlink(),'fresh model output')
    print(json.dumps(emit(a.out,a.corrupt_cyclotomic),indent=2))
```

phase236_finite_check.g

```
Need236:=function(ok,msg) if not ok then Error(Concatenation("PHASE236_REJECT ",msg)); fi;
 ↪ end;
IP236:=function(rr)
  local subsets,hs,I,J;
  subsets:=Combinations([1..3]); hs:=List(subsets,I->Group(List(I,i->rr[i]),()));
  for I in [1..8] do for J in [1..8] do
    if Intersection(hs[I],hs[J])<>hs[Position(subsets,Intersection(subsets[I],subsets[J]))]
     ↪ then
      return false;
    fi;
  od;od;return true;
end;
Run236:=function()
local row,p,m,s,beta,sigma,tau,P,A,ss,i,u,v,rr,G,par,a,b,points,x,y,r0,r1,r2;
for row in PHASE236_DATA do
  p:=row.p; m:=row.m; s:=row.s; beta:=row.beta; sigma:=row.sigma; tau:=row.tau;
  P:=Group(s,beta); A:=Subgroup(P,Orbit(Group(beta),s,OnPoints));
  Need236(Size(A)=p^(m-1) and IsAbelian(A),"abelian lattice order");
  Need236(Size(P)=p^m and not IsAbelian(P),"nonabelian kernel order");
  Need236(Order(beta)=p and Order(sigma)=2 and Order(tau)=2 and
   ↪ Comm(sigma,tau)=(),"complement");
  Need236(s^sigma=s and beta^sigma=beta^-1 and s^tau=s^-1 and beta^tau=beta,"source
   ↪ automorphisms");
  ss:=[s]; for i in [2..p-1] do Add(ss,Comm(ss[i-1],beta)); od;
  Need236(List(ss,Order)=row.moduli,"every source generator order");
  for u in ss do for v in ss do Need236(Comm(u,v)=(),"source abelian relators");od;od;
  Need236(Comm(ss[p-1],beta)=Product([1..p-1],i->ss[i]^(-Binomial(p,i))),"last source
   ↪ commutator");
  rr:=[s*tau*sigma,beta*tau*sigma,sigma]; G:=Group(rr);
  Need236(ForAll(rr,x->Order(x)=2),"marked involutions");
  Need236(Comm(rr[1],rr[3])=(),"string relation");
  Need236([Order(rr[1]*rr[2]),Order(rr[2]*rr[3])]=[p,2*p],"type");
  Need236(Size(G)=4*p^m,"full generation and order");
  Need236(IP236(rr),"all 64 intersection equalities");
```

```
  Need236(NilpotencyClassOfGroup(P)=m-1,"maximal class");
  Print("CYCLOTOMIC p=",p," m=",m," order=",Size(G)," moduli=",row.moduli," IP=64 PASS\n");
od;
# Independent affine models for the abelian classification, including unequal exponents.
for par in [[3,1,1],[3,1,2],[3,2,2],[5,1,2]] do
  p:=par[1]; a:=par[2]; b:=par[3]; points:=Cartesian([0..p^b-1],[0..p^a-1]);
  x:=PermList(List(points,v->Position(points,[(v[1]+1) mod p^b,v[2]])));
  y:=PermList(List(points,v->Position(points,[v[1],(v[2]+1) mod p^a])));
  r0:=PermList(List(points,v->Position(points,[v[1],(-v[2]) mod p^a])));
  r2:=PermList(List(points,v->Position(points,[(-v[1]) mod p^b,(-v[2]) mod p^a])));
  r1:=x*y*r2;
  rr:=[r0,r1,r2]; G:=Group(rr);
  Need236(ForAll(rr,r->Order(r)=2),"abelian marked involutions");
  Need236(Size(G)=4*p^(a+b),"abelian group order");
  Need236([Order(r0*r1),Order(r1*r2)]=[2*p^a,p^b],"abelian type");
  Need236(IP236(rr),"abelian intersection property");
  Need236((x*y*r0)^2=x^2 and x^2<>(),"literal source marking defect");
  Print("ABELIAN p=",p," a=",a," b=",b," order=",Size(G)," IP=64 BAD_MARKING_REJECTED\n");
od;
Print("PHASE236_FINITE_PASS\n");
end;
Run236();
QUIT;
```

phase248_presentation_check.g

```
# Independent literal source-presentation replay, without cyclotomic matrices.
Check248:=function(ok,s) if not ok then Error(Concatenation("P248_REJECT ",s));fi;end;;
IP248:=function(rr)
 local subsets,hs,i,j;
 subsets:=Combinations([1..3]);hs:=List(subsets,x->Group(rr{x},One(rr[1])));
 for i in [1..8] do for j in [1..8] do
  if Intersection(hs[i],hs[j])<>hs[Position(subsets,Intersection(subsets[i],subsets[j]))]
   ↪  then return false;fi;
 od;od;return true;
end;;
Run248:=function()
 local par,p,m,n,e,r,F,x,b,rels,i,j,U,iso,P,ss,beta,sigma,tau,K,G,a,z,rs,moduli;
 for par in [[3,3],[3,6],[5,3],[5,5],[11,3]] do
  p:=par[1];m:=par[2];n:=m-1;e:=QuoInt(n-1,p-1)+1;r:=((n-1) mod (p-1))+1;
  F:=FreeGroup(p);x:=GeneratorsOfGroup(F){[1..p-1]};b:=GeneratorsOfGroup(F)[p];
  moduli:=Concatenation(List([1..r],i->p^e),List([r+1..p-1],i->p^(e-1)));
  rels:=Concatenation(List([1..p-1],i->x[i]^moduli[i]),[b^p]);
  for i in [1..p-1] do for j in [i+1..p-1] do if j<=p-1 then
   ↪  Add(rels,Comm(x[i],x[j]));fi;od;od;
  for i in [1..p-2] do Add(rels,Comm(x[i],b)*x[i+1]^-1);od;
  Add(rels,Comm(x[p-1],b)*Product([1..p-1],i->x[i]^Binomial(p,i)));
  U:=F/rels;Check248(Size(U)=p^m,"exact presented order");
  iso:=IsomorphismPermGroup(U);Check248(iso<>fail,"permutation isomorphism");P:=Image(iso);
  ss:=List(GeneratorsOfGroup(U){[1..p-1]},x->Image(iso,x));beta:=Image(iso,GeneratorsOfGrou⌋
   ↪  p(U)[p]);
  Check248(List(ss,Order)=moduli,"individual source orders");
  sigma:=GroupHomomorphismByImages(P,P,[ss[1],beta],[ss[1],beta^-1]);
  tau:=GroupHomomorphismByImages(P,P,[ss[1],beta],[ss[1]^-1,beta]);
  Check248(sigma<>fail and tau<>fail and IsBijective(sigma) and IsBijective(tau),"literal
   ↪  maps extend to automorphisms");
  K:=Group(sigma,tau);Check248(Size(K)=4,"Klein complement");
  G:=SemidirectProduct(K,P);a:=Embedding(G,1);z:=Embedding(G,2);
  rs:=[Image(z,ss[1])*Image(a,tau)*Image(a,sigma),Image(z,beta)*Image(a,tau)*Image(a,sigma)⌋
   ↪  ,Image(a,sigma)];
  Check248(Size(Group(rs))=4*p^m and ForAll(rs,x->Order(x)=2),"marked generation");
  Check248([Order(rs[1]*rs[2]),Order(rs[2]*rs[3])]=[p,2*p],"exact type");
  Check248(Comm(rs[1],rs[3])=One(G) and IP248(rs),"all marked intersections");
  Print("LITERAL_PRESENTATION ",p," ",m," ORDER ",Size(U)," MODULI ",moduli," IP64 PASS\n");
 od;
 # The prime-index hypothesis is necessary: a composite-index subgroup can
 # meet another subgroup nontrivially without being contained in it.
 G:=ElementaryAbelianGroup(8);x:=GeneratorsOfGroup(G);P:=Group(x{[1,2]});K:=Group(x{[1,3]});
 Check248(Index(P,TrivialSubgroup(P))=4 and not IsSubgroup(K,P) and
  ↪  Size(Intersection(P,K))=2,"composite-index false weakening");
 Print("COMPOSITE_INDEX_FALSE_WEAKENING_REJECTED\nPHASE248_PRESENTATION_PASS\n");
end;;
```

```
Run248();
QUIT_GAP(0);
```

cyclotomic_carries.g

```
# Finite diagnostics of the uniform written construction, not proof premises.
Need := function(ok,msg) if not ok then Error(Concatenation("L007_REJECT: ",msg)); fi; end;
CarryModel := function(p,n,even)
 local e,r,mods,aa,ag,ss,i,T,ims,zz,m,s,z,tm,dd,ea,ec,cc,cs,ts,dm,
       cp,cpgen,act,E,beta0,em,eb,N,q,P,beta,sp,zp,sig,G,ep,es,a,b,gens,xx;
 e:=QuoInt(n-1,p-1)+1; r:=(n-1) mod (p-1)+1;
 Need(n mod 2=0 and n>=4,"even module length >=4");
 if even then Need(e>=2,"source even extension e>=2");fi;
 mods:=Concatenation(List([1..r],i->p^e),List([r+1..p-1],i->p^(e-1)));
 aa:=AbelianGroup(IsPcGroup,Filtered(mods,k->k>1));ag:=GeneratorsOfGroup(aa);
 ss:=List([1..p-1],i->One(aa));
 for i in [1..Length(ag)] do ss[i]:=ag[i];od;
 ims:=[];
 for i in [1..p-2] do Add(ims,ss[i]*ss[i+1]);od;
 Add(ims,ss[p-1]*Product([1..p-1],j->ss[j]^(-Binomial(p,j))));
 T:=GroupHomomorphismByImages(aa,aa,ss,ims);
 Need(T<>fail and IsBijective(T) and Order(T)=p,"actual cyclotomic automorphism");
 zz:=ss[r]^(p^(e-1));
 if not even then
  m:=aa;s:=ss[1];z:=zz;tm:=T;
 else
  cc:=CyclicGroup(IsPcGroup,p^2);dd:=DirectProduct(aa,cc);
  ea:=Embedding(dd,1);ec:=Embedding(dd,2);cs:=Image(ec,GeneratorsOfGroup(cc)[1]);
  s:=Image(ea,ss[1])*cs;z:=Image(ea,zz);
  dm:=GroupHomomorphismByImages(dd,dd,
      Concatenation(List(ag,g->Image(ea,g)),[cs]),
      Concatenation(List(ag,g->Image(ea,Image(T,g))),[cs]));
  Need(dm<>fail and IsBijective(dm),"product automorphism");
  m:=Group(List([0..p-1],i->Image(dm^i,s)));
  Need(Size(m)=p^(n+1) and z in m,"complete module fibre");
  tm:=GroupHomomorphismByImages(m,m,GeneratorsOfGroup(m),
     List(GeneratorsOfGroup(m),g->Image(dm,g)));
 fi;
 Need(Order(z)=p and Image(tm,z)=z,"nonzero fixed carry");
 cp:=CyclicGroup(IsPcGroup,p^2);cpgen:=GeneratorsOfGroup(cp)[1];
 act:=GroupHomomorphismByImages(cp,Group(tm),[cpgen],[tm]);
 E:=SemidirectProduct(cp,act,m);em:=Embedding(E,2);eb:=Embedding(E,1);
 beta0:=Image(eb,cpgen);
 # GAP's action convention is verified by the exact generator equation.
 Need(ForAll(GeneratorsOfGroup(m),g->Image(em,g)^beta0=Image(em,Image(tm,g))),"right
  ↪  conjugation convention");
 N:=Group(beta0^p*Image(em,z)^-1);Need(Size(N)=p and IsNormal(E,N),"central relation
  ↪  kernel");
 q:=NaturalHomomorphismByNormalSubgroup(E,N);P:=Image(q);
 beta:=Image(q,beta0);sp:=Image(q,Image(em,s));zp:=Image(q,Image(em,z));
 Need(Size(P)=p*Size(m) and beta^p=zp and Order(zp)=p,"extension order/injection/carry");
 Need(Size(Group(beta,sp))=Size(P),"two generators of kernel");
 sig:=GroupHomomorphismByImages(P,P,[beta,sp],[beta^-1,sp]);
 Need(sig<>fail and IsBijective(sig) and Order(sig)=2,"reflection involution");
 G:=SemidirectProduct(Group(sig),P);ep:=Embedding(G,2);es:=Embedding(G,1);
 a:=Image(ep,beta*sp);b:=Image(ep,sp^-1)*Image(es,sig);
 Need(Size(Group(a,b))=Size(G),"marked generation");
 Need([Order(a),Order(b),Order(a*b)]=[p^2,2*p^e,2],"exact marker orders");
 Need(Size(Intersection(Group(a),Group(b)))=1,"cyclic intersection");
 Need(GroupHomomorphismByImages(G,G,[a,b],[a^-1,b^-1])=fail,"no simultaneous inversion");
 Need(Order(a)*Order(b)<Size(G),"non-tightness");
 Print("CARRY ",rec(p:=p,n:=n,even_extension:=even,order:=Size(G),type:=[Order(a),Order(b)]
  ↪  ,intersection:=1,reversal:=false),"\n");
 return true;
end;
for par in [[3,4,false],[3,6,false],[5,4,false],[7,4,false],[5,6,false],
 [3,4,true],[3,6,true],[5,6,true],[7,8,true]] do
 CarryModel(par[1],par[2],par[3]);
od;
QUIT_GAP(0);
```

regular_byproducts.g

```
# Exact validation of the corrected four-family regular scope.
Need:=function(ok,msg) if not ok then Error(Concatenation("L007_REJECT: ",msg));fi;end;
IP:=function(rs,target)
 local g,subs,I,J,ij;
 g:=Group(rs);Need(Size(g)=target,"full regular generation/order");
 Need(ForAll(rs,r->Order(r)=2) and rs[1]*rs[3]=rs[3]*rs[1],"string involutions");
 subs:=Combinations([1..3]);
 for I in subs do for J in subs do
  ij:=Intersection(I,J);
  Need(Intersection(Subgroup(g,rs{I}),Subgroup(g,rs{J}))=Subgroup(g,rs{ij}),"full subset
   ↪ IP");
 od;od;
 return [Order(rs[1]*rs[2]),Order(rs[2]*rs[3])];
end;
CyclotomicRegular:=function(p,n)
 local
  ↪ e,r,mods,A,gg,ss,i,ims,T,P,ep,eb,s,beta,sig,tau,K,R,er,ek,sg,tg,bg,sp,rs,a,b,c,ty1,ty2;
 e:=QuoInt(n-1,p-1)+1;r:=(n-1) mod (p-1)+1;
 mods:=Concatenation(List([1..r],i->p^e),List([r+1..p-1],i->p^(e-1)));
 A:=AbelianGroup(IsPcGroup,Filtered(mods,k->k>1));gg:=GeneratorsOfGroup(A);
 ss:=List([1..p-1],i->One(A));for i in [1..Length(gg)] do ss[i]:=gg[i];od;
 ims:=List([1..p-2],i->ss[i]*ss[i+1]);
 Add(ims,ss[p-1]*Product([1..p-1],j->ss[j]^(-Binomial(p,j))));
 T:=GroupHomomorphismByImages(A,A,ss,ims);Need(T<>fail and Order(T)=p,"cyclotomic action");
 P:=SemidirectProduct(Group(T),A);ep:=Embedding(P,2);eb:=Embedding(P,1);
 s:=Image(ep,ss[1]);beta:=Image(eb,T);
 sig:=GroupHomomorphismByImages(P,P,[s,beta],[s,beta^-1]);
 tau:=GroupHomomorphismByImages(P,P,[s,beta],[s^-1,beta]);
 Need(sig<>fail and tau<>fail,"two module
  ↪ involutions");K:=Group(sig,tau);Need(Size(K)=4,"independent complement");
 R:=SemidirectProduct(K,P);er:=Embedding(R,2);ek:=Embedding(R,1);
 sp:=Image(er,s);bg:=Image(er,beta);sg:=Image(ek,sig);tg:=Image(ek,tau);
 rs:=[sp*tg*sg,bg*tg*sg,sg];ty1:=IP(rs,4*p^(n+1));Need(ty1=[p,2*p],"constant type");
 a:=bg*sp;b:=sp^-1*sg;c:=tg*sg*sp;
 Need(a^c=a^-1 and b^c=b^-1,"literal second-mark reversal");
 ty2:=IP([a*c,c,c*b],4*p^(n+1));Need(ty2=[p,2*p^e],"variable type");
 Print("REGULAR_CYC
  ↪ ",rec(p:=p,n:=n,order:=Size(R),types:=[ty1,ty2],IP_pairs_per_marking:=64),"\n");
end;
MixedRegular:=function(p)
 local f,col,H,hs,x,v,c,theta,C,q,a,E,eh,eb,b0,N,proj,G,xg,bg,delta,R,eg,ed,xx,bb,dd,ty;
 f:=FreeGroup(4);col:=SingleCollector(f,[p,p,p,p]);
 SetPower(col,1,f.4);
 SetConjugate(col,2,1,f.2*f.3^-1);UpdatePolycyclicCollector(col);
 H:=GroupByRws(col);hs:=GeneratorsOfGroup(H);x:=hs[1];v:=hs[2];c:=hs[3];
 Need(Size(H)=p^4 and Comm(x,v)=c and Order(x)=p^2,"mixed Heisenberg normal form");
 theta:=GroupHomomorphismByImages(H,H,[x,v],[v^-1*x^-1,v]);
 Need(theta<>fail and Order(theta)=2*p,"mixed action");
 C:=CyclicGroup(IsPcGroup,2*p);a:=GeneratorsOfGroup(C)[1];
 q:=GroupHomomorphismByImages(C,Group(theta),[a],[theta]);E:=SemidirectProduct(C,q,H);
 eh:=Embedding(E,2);eb:=Embedding(E,1);b0:=Image(eb,a);N:=Group(b0^2*Image(eh,v)^-1);
 Need(Size(N)=p and IsNormal(E,N),"square-root central kernel");
 proj:=NaturalHomomorphismByNormalSubgroup(E,N);G:=Image(proj);xg:=Image(proj,Image(eh,x));
  ↪ bg:=Image(proj,b0);
 Need(Size(G)=2*p^4 and Order(xg*bg)=2,"mixed rotation group");
 delta:=GroupHomomorphismByImages(G,G,[xg,bg],[xg^-1,bg^-1]);
 Need(delta<>fail and Order(delta)=2,"mixed reversal");
 R:=SemidirectProduct(Group(delta),G);eg:=Embedding(R,2);ed:=Embedding(R,1);
 xx:=Image(eg,xg);bb:=Image(eg,bg);dd:=Image(ed,delta);
 ty:=IP([xx*dd,dd,dd*bb],4*p^4);Need(ty=[p^2,2*p],"mixed type");
 Print("REGULAR_MIXED ",rec(p:=p,order:=Size(R),type:=ty,IP_pairs:=64),"\n");
end;
for pp in [3,5,7] do
 CyclotomicRegular(pp,2);CyclotomicRegular(pp,3);MixedRegular(pp);
od;
QUIT_GAP(0);
```

small_order_probe.g

```
# Exploratory exact scan. The accepted final boundary has not yet been fixed.
Print("GAP ",GAPInfo.Version,"\n");
for sz in [54,162,1250] do
  found:=0; pairs:=0; cnt:=NrSmallGroups(sz);
  Print("ORDER ",sz," catalog ",cnt,"\n");
  for id in [1..cnt] do
    g:=SmallGroup(sz,id);
    xs:=Filtered(List(ConjugacyClasses(g),Representative),x->Order(x)>2 and
     ↪  IsOddInt(Order(x)));
    ts:=Filtered(Elements(g),t->Order(t)=2);
    for x in xs do
      c:=Centralizer(g,x);
      reps:=List(Orbits(c,ts,OnPoints),o->o[1]);
      for t in reps do
        y:=x^-1*t;
        if Order(y)>2 and Size(Intersection(Group(x),Group(y)))=1 and Order(x)*Order(y)<sz
         ↪  and Size(Group(x,t))=sz then
          pairs:=pairs+1;
          h:=GroupHomomorphismByImages(g,g,[x,y],[x^-1,y^-1]);
          if h=fail then Print("CHIRAL ",sz," ",id,"
           ↪  ",[Order(x),Order(y)],"\n");found:=found+1;fi;
          if sz=1250 and Order(x)=5 and Order(y)=50 then Print("REMARK TYPE FOUND\n");fi;
        fi;
      od;
    od;
  od;
  Print("RESULT ",sz," non-tight pair orbits ",pairs," chiral ",found,"\n");
od;
QUIT_GAP(0);
```

modularity_probe.g

```
# Exploratory, not part of the frozen proof boundary.
for q in [27,81,243] do
 for id in [1..NrSmallGroups(q)] do
  g:=SmallGroup(q,id);
  if not IsAbelian(g) and ForAny(NormalSubgroups(g),n->IsCyclic(n) and
   ↪  IsCyclic(FactorGroup(g,n))) then
   subs:=List(ConjugacyClassesSubgroups(g),Representative);
   cyc:=Filtered(subs,IsCyclic);
   for u in cyc do for v0 in cyc do for v in ConjugacyClassSubgroups(g,v0) do
    h:=ClosureGroup(u,v);
    if Size(h)*Size(Intersection(u,v))<>Size(u)*Size(v) then
     Print("NONPERMUTABLE ",[q,id]," subgroup orders ",[Size(u),Size(v)]," generated
      ↪  ",Size(h),"\n");
     QUIT_GAP(0);
    fi;
   od; od; od;
  fi;
 od;
 Print("checked ",q,"\n");
od;
QUIT_GAP(0);
```

central_lift.g

```
# Central cyclic lift of the odd-exponent construction in arXiv:2508.20654.
# Run: gap -q research/leemans/central_lift.g
# The independent checks use GAP's exact finite polycyclic group algorithms.

VerifyCentralLift := function(p, m)
  local r, f, col, i, P, pc, beta, s, sgimages, sig, Aut2, G0,
        embP, embSig, b0, s0, sig0, a0, c0, C, D, eG, eC, x,
        a, b, H, invmap, K, phi, target, result, t, hpc;
  if not IsPrimeInt(p) or p < 5 or m < 6 or m mod 2 <> 0 or m > p+1 then
    Error("Require odd prime p >= 5 and even 6 <= m <= p+1");
  fi;
```

```
  r := m-2;
  f := FreeGroup(r+1);
  col := SingleCollector(f, List([1..r+1], i -> p));
  SetPower(col,1,f.(r+1));
  for i in [2..r] do
    SetConjugate(col,i,1,f.(i)*f.(i+1));
  od;
  UpdatePolycyclicCollector(col);
  P := GroupByRws(col);
  pc := Pcgs(P);
  beta := pc[1];
  s := pc[2];
  if Size(P) <> p^(m-1) then Error("Wrong base p-group order"); fi;
  sig := GroupHomomorphismByImages(P,P,[beta,s],[beta^-1,s]);
  if sig=fail or not IsBijective(sig) or Order(sig) <> 2 then
    Error("Base involutory automorphism fails");
  fi;
  Aut2 := Group(sig);
  G0 := SemidirectProduct(Aut2,P);
  embSig := Embedding(G0,1);
  embP := Embedding(G0,2);
  b0 := Image(embP,beta);
  s0 := Image(embP,s);
  sig0 := Image(embSig,sig);
  a0 := b0*s0;
  c0 := s0^-1*sig0;
  if Size(Group(a0,c0)) <> 2*p^(m-1) then Error("Base generation fails"); fi;
  if [Order(a0),Order(c0),Order(a0*c0)] <> [p^2,2*p,2] then
    Error("Base orders fail");
  fi;
  if Size(Intersection(Group(a0),Group(c0))) <> 1 then Error("Base IP fails"); fi;
  if GroupHomomorphismByImages(G0,G0,[a0,c0],[a0^-1,c0^-1]) <> fail then
    Error("Base is reflexible");
  fi;
  target := CyclicGroup(IsPcGroup,p);
  phi := GroupHomomorphismByImages(G0,target,[a0,c0],
            [GeneratorsOfGroup(target)[1],GeneratorsOfGroup(target)[1]^-1]);
  if phi=fail or not IsSurjective(phi) then Error("Fiber map fails"); fi;
  C := CyclicGroup(IsPcGroup,p^2);
  D := DirectProduct(G0,C);
  eG := Embedding(D,1);
  eC := Embedding(D,2);
  x := Image(eC,GeneratorsOfGroup(C)[1]);
  a := Image(eG,a0)*x;
  b := Image(eG,c0)*x^-1;
  H := Group(a,b);
  if Size(H) <> 2*p^m then Error("Lift size fails"); fi;
  if [Order(a),Order(b),Order(a*b)] <> [p^2,2*p^2,2] then
    Error("Lift orders fail");
  fi;
  if Size(Intersection(Group(a),Group(b))) <> 1 then Error("Lift IP fails"); fi;
  K := Group(b^(2*p));
  if Size(K) <> p or not IsNormal(H,K) then Error("Lift kernel fails"); fi;
  invmap := GroupHomomorphismByImages(H,H,[a,b],[a^-1,b^-1]);
  if invmap <> fail then Error("Lift is reflexible"); fi;
  result := rec(p:=p, m:=m, base_order:=Size(G0), order:=Size(H),
    type:=[Order(a),Order(b)], product_order:=Order(a*b),
    cyclic_intersection_order:=Size(Intersection(Group(a),Group(b))),
    lift_kernel_order:=Size(K), reversal_homomorphism_exists:=invmap<>fail,
    tight:=Size(H)=Order(a)*Order(b));
  Print(result,"\n");
  return result;
end;

Print("GAP ",GAPInfo.Version,"\n");
VerifyCentralLift(5,6);;
VerifyCentralLift(7,6);;
VerifyCentralLift(7,8);;
QUIT;
```

verify_lift_normalforms.py

```
"""Independent explicit normal-form verification of the small central lift.

No symbolic group package or unproved presentation-consistency assumptions.
The ambient group is an elementary abelian module extended by beta and sigma,
times C_(p^2), with multiplication encoded by its two matrix actions.
"""

from collections import deque
import argparse
import json


def verify(p=5, r=4):
    assert p >= 5 and r % 2 == 0 and 4 <= r <= p - 1
    assert all(p % d for d in range(2, int(p**0.5) + 1)), "p must be prime"
    # Vectors use the cyclic module basis e_i = (T-I)^i e_0.
    # T^n=(I+N)^n, with integer generalized binomial coefficients.
    def binomial(n, k):
        z = 1
        for j in range(k):
            z = z * (n - j) // (j + 1)
        return z

    def T(v, n):
        return tuple(sum(binomial(n, j - i) * v[i]
                         for i in range(j + 1)) % p for j in range(r))

    zero = (0,) * r
    e0 = (1,) + (0,) * (r - 1)
    er = (0,) * (r - 1) + (1,)
    # S(e_i)=(T^-1-I)^i(e_0); this satisfies STS=T^-1.
    cols = [e0]
    for i in range(1, r):
        previous = cols[-1]
        transformed = T(previous, -1)
        cols.append(tuple((transformed[j] - previous[j]) % p
                          for j in range(r)))

    def S(v):
        return tuple(sum(v[i] * cols[i][j] for i in range(r)) % p
                     for j in range(r))

    for i in range(r):
        ei = tuple(int(j == i) for j in range(r))
        assert S(S(ei)) == ei
        assert S(T(S(ei), 1)) == T(ei, -1)
        assert T(ei, p) == ei
    assert S(er) == tuple((-z) % p for z in er)
    assert T(er, 1) == er

    # Elements are (v,k,epsilon,j) = v beta^k sigma^epsilon x^j,
    # 0<=k<p, epsilon in {0,1}, 0<=j<p^2, beta^p=e_(r-1).
    identity = (zero, 0, 0, 0)

    def mul(g, h):
        v, k, epsilon, j = g
        w, l, delta, q = h
        action = T(S(w) if epsilon else w, -k)
        combined = k + (-l if epsilon else l)
        carry, exponent = divmod(combined, p)
        vector = tuple((v[i] + action[i] + (carry if i == r-1 else 0)) % p
                       for i in range(r))
        return (vector, exponent, (epsilon + delta) % 2, (j + q) % (p*p))

    def power(g, n):
        ans = identity
        while n:
            if n & 1:
                ans = mul(ans, g)
            g = mul(g, g)
            n //= 2
        return ans
```

```
    def cyclic(g):
        ans = {identity}
        x = g
        while x != identity:
            assert x not in ans, "Multiplication did not define a finite cyclic group"
            ans.add(x)
            x = mul(x, g)
        return ans

    beta = (zero, 1, 0, 0)
    sigma = (zero, 0, 1, 0)
    s1 = (e0, 0, 0, 0)
    assert power(beta, p) == (er, 0, 0, 0)
    assert mul(mul(sigma, beta), sigma) == power(beta, p*p-1)
    assert mul(mul(sigma, s1), sigma) == s1
    a = (T(e0, -1), 1, 0, 1)
    b = (tuple((-z) % p for z in e0), 0, 1, p*p-1)
    aa, bb = cyclic(a), cyclic(b)
    assert len(aa) == p*p
    assert len(bb) == 2*p*p
    assert aa.intersection(bb) == {identity}
    assert power(mul(a, b), 2) == identity
    seen = {identity}
    queue = deque([identity])
    while queue:
        g = queue.popleft()
        for h in (a, b):
            nxt = mul(g, h)
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    expected = 2*p**(r+2)
    assert len(seen) == expected
    projected = {(v, k, epsilon) for v, k, epsilon, j in seen}
    kernel = {(v, k, epsilon, j) for v, k, epsilon, j in seen
              if v == zero and k == 0 and epsilon == 0}
    assert len(projected) == 2*p**(r+1)
    assert len(kernel) == p
    assert kernel == cyclic(power(b, 2*p))
    # The fiber equation is exact, not just a subgroup order coincidence.
    assert all(j % p == v[0] for v, k, epsilon, j in seen)
    return {"p": p, "r": r, "m": r+2, "group_order": len(seen),
            "base_order": len(projected), "generator_orders": [len(aa), len(bb)],
            "product_order": len(cyclic(mul(a, b))),
            "cyclic_intersection_order": len(aa & bb), "kernel_order": len(kernel),
            "note": "Chirality follows from the proved invariant-kernel argument; no
             ↪  automorphism enumeration used."}


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--p", type=int, default=5)
    parser.add_argument("--r", type=int, default=4)
    args = parser.parse_args()
    print(json.dumps(verify(args.p, args.r), indent=2))
```